



\$derived.by: The Block-Body Form

Q38

SVELTE 5 RUNES \$DERIVED.BY COMPUTED VALUES

INTERVIEW QUESTION

Question. How does the `$derived.by` rune differ from a standard `$derived` expression, and when is it necessary?

ANSWER. `$derived` and `$derived.by` are the same mechanism wearing two syntactic shapes. The plain form `$derived(expr)` takes a single expression; `.by` takes a zero-argument function whose body is a block of statements, and returns whatever the block returns. Under the hood they compile to the same memoized, lazily-recomputed, dependency-tracked signal — the dependency tracking, caching, and laziness rules are identical to Q37.

`.by` is necessary in exactly one situation: **when the computation cannot be written as a single expression**. The rule is prescriptive, not stylistic — use `$derived(expr)` whenever you can, and reach for `$derived.by(() => { ... })` only when you genuinely need statements. The four common triggers are: a `let` for an intermediate variable, a `for` or `for...of` loop that accumulates a result, a `try/catch` around something that may throw, or several sequential calculations that read more clearly as steps than as one tangled ternary. If you find yourself writing `$derived.by(() => { return expr; })` — a function whose body is just a single return — you have written the wrong form; collapse it back to `$derived(expr)`.

```
<script>
  let items = $state([10, 20, 30]);
  let tax = $state(0.2);

  // Single expression → plain $derived.
  let count = $derived(items.length);

  // Needs statements (a loop + an intermediate let) →
  $derived.by.
  let totalWithTax = $derived.by(() => {
    let subtotal = 0;
    for (const item of items) subtotal += item;
    return subtotal * (1 + tax);
  });
</script>

<p>{count} items, total: {totalWithTax}</p>
```

❶ **Summary.** `$derived(expr)` and `$derived.by(() => { ... })` are the same memoized signal under the hood — `.by` exists only for computations that need statements (loops, intermediate `let` s, `try/catch`, multi-step logic). Reach for plain `$derived` first; switch to `.by` only when an expression genuinely cannot carry the work.

SAME MEMOIZED SIGNAL UNDERNEATH

`$derived(expr)`

one expression

REACH FOR THIS FIRST

`$derived.by(() => {
... })`

`let subtotal = 0;`

`for (...) subtotal
+= ...;`

`return subtotal *
(1 + tax);`

STATEMENTS ALLOWED